

DRIFT, COST, RELEASE, CAPACITY, GOVERNANCE & COMPLIANCE

Study Guide — Operating and Governing Production AIP Functions and Chatbots

Week 6 · Day 2 · Prepared by Rajesh Pasham

Program Context

Item	Detail
Session Type	Concepts and architecture lecture — no live platform work today
Audience	India and Europe batches together — this session is common to all four batches
This Week's Focus	Production AIP, LLMOps & Governance — hardening the Copilots, chatbots, and agent workflows built in Weeks 2–5
This Week's Deliverable (Day 3)	A Production Readiness Package: an evaluation suite, an observability dashboard, and a release plan

Bridge From Day 1

Day 1's AIP Evals and Foundry Observability gave you a way to know whether your functions and chatbots are behaving correctly, right now. That is a signal, not an operation. Today turns that signal into action: what do you do when a function's behavior quietly shifts? How do you roll out a changed prompt without risking every user at once? How do you plan for a traffic spike before it happens? And who is allowed to change any of this in the first place, with what trail left behind?

Today's six topics — drift monitoring, cost management, release strategy, capacity planning, AI governance and responsible AI, and audit and compliance — complete the production-readiness bar Day 1 started. We do not re-teach the LLMOps lifecycle, prompt versioning, AIP Evals, or Foundry Observability in depth here; we assume that vocabulary and build directly on it, using the same Batch 1 through 4 functions and chatbots as worked examples.

Takeaway

Nothing new is being introduced in isolation today. Every topic below is Day 1's evaluation and observability foundation, extended into an operational practice (drift, cost, release, capacity) or a governance practice (AI governance, audit and compliance) built on Foundry primitives you already used in Week 1.

Learning Objectives

By the end of today, you will be able to:

- Explain what drift means for an LLM-powered function and why it can occur with no code change
- Describe cost management as an LLMOps discipline and identify which block types in a function are costliest to run at scale
- Describe a staged release strategy for a prompt or model change
- Explain capacity planning for an LLM-powered system, anticipating a traffic or load spike before it arrives
- Apply Foundry's confirmed governance primitives — Roles, Markings, Security Policies — to an AI-specific risk

- Apply Foundry's Data Lineage and Audit Trails to trace why an AIP Chatbot took a specific action

Topic 1 — Drift Monitoring

1. What Drift Means for an LLM-Powered Function

- **The function's code and prompt can stay identical while its judgment shifts** — a platform-level model update, or a subtle change in how the underlying model interprets an unchanged prompt, can shift a function's output quality with absolutely nothing in your Ontology or AIP Logic canvas showing as changed.
- **Concretely, in a function you've already built** — if FraudRiskAssessment's (Batch 1) underlying model is swapped or its prompt is edited elsewhere, the same claim that used to evaluate recommendedFraudAction = OpenInvestigation at claimAmount = 48,000 could start evaluating Escalate instead, with no code change flagged anywhere — that is the drift this discipline exists to catch.
- **Drift is why Day 1's evaluation has to be continuous, not a one-time gate** — AIP Evals gives you the mechanism: re-running an Evaluation Suite's Test Cases periodically, not just once at launch, and comparing Metrics across those runs over time is literally how drift gets detected in practice.

2. How a Team Actually Catches Drift

Combining Day 1's building blocks into an ongoing discipline — not a new named Palantir feature, a practice run on confirmed tools:

- **Re-run the Evaluation Suite on a schedule, not just on change** — the same Evaluation Suite built for RiskScoreRefresh (Batch 2) — Target Function RiskScoreRefresh, its Test Cases, its Evaluation Function — gets re-run weekly or after any platform-level model update notice, and its pass-rate Metric gets compared against last week's, not just against a pass/fail threshold in isolation.
- **Watch Foundry Observability's confirmed metrics for a trend, not just a spike** — Foundry Observability's confirmed per-resource success/failure counts and P95 execution duration for ComplianceReadinessCheck (Batch 3) can surface a slow behavioral shift indirectly — a rising failure rate or a widening duration spread over 30 days is a signal worth investigating even before a user complains.
- **Export traces for a longer-horizon comparison** — exporting a month of the Fraud Detection Chatbot's (Batch 1) traces, as Day 1's Analysis capability described, and comparing how often it recommends OpenInvestigation this month against last month is a concrete way to catch drift that single-run evals would miss.

3. Drift Monitoring in Practice: A Worked Example

Tracing a quiet shift in CoverageRecommendationAssessment (Batch 4) from first symptom to confirmed drift:

1. A support rep notices CoverageRecommendationAssessment is recommending Escalate more often than it used to

No alert fired, no error appeared in the logs — a clean error log does not mean correct behavior, and this is exactly that gap surfacing.

2. The team re-runs CoverageRecommendationAssessment's Evaluation Suite against its existing Test Cases

Including the recommendationTier test cases built around the €5,000 High-Value Renewal threshold — and compares this run's pass rate against the Metrics recorded when the function was last validated.

3. The comparison shows a real shift, not noise

Several Test Cases that previously evaluated recommendationTier = Low are now coming back Medium — consistent across repeated runs, which is what rules out ordinary LLM non-determinism as the explanation.

4. The team traces the cause and decides on a response

A platform-level model update is identified as the likely trigger; the team either adjusts the prompt to compensate and re-validates, or escalates to Palantir support — either way, this becomes a Release Strategy decision (Topic 3).

Common Mistake

Assuming a function that passed Day 1's evals last month is still performing the same way today. A platform-level model update can shift a function's judgment with zero code or prompt change flagged — that is drift, and it is why evaluation and monitoring are continuous, not a one-time gate.

Topic 2 — Cost Management

4. Why Cost Management Is Its Own Topic

- **An LLM-powered function has a per-call cost a deterministic function does not** — Foundry Observability's confirmed metrics (success/failure counts, P95 execution duration, execution counts) tell you how often and how long a function runs, and current Foundry Observability also surfaces token usage and prompt visibility for LLM calls — but the cost-per-call discipline on top of that is one your team builds and watches, the same way you'd watch any other cost that scales with traffic.
- **Not every block in a function costs the same** — within RiskScoreRefresh (Batch 2), the deterministic riskScoreStale calculation — a plain Create variable expression over assessmentDate and isHighRiskApplication — runs at effectively negligible cost, while the function's Use LLM risk-judgment block, weighing riskScore, riskCategory, applicationSummary, and riskFactors, is the block actually consuming tokens on every single call.
- **The same distinction holds across every function you've built** — FraudRiskAssessment's (Batch 1) thresholdCrossed check is a deterministic Create variable block; its fraud-judgment Use LLM block is the costly one. ComplianceReadinessCheck's (Batch 3) regulatorySignOffRequired check is deterministic; its compliance-judgment Use LLM block is the costly one. Knowing which block is which is the first step in cost management, not an afterthought.

5. What a Cost Management Discipline Actually Looks Like

- **Track token volume and cost per call in your own dashboard** — mirroring a well-known pattern in production LLM deployments more broadly: production teams commonly export token usage data — available through Foundry Observability for LLM calls — to a streaming dataset and build a Workshop dashboard tracking cost per call, cost per function, and cost per batch over time, the same Analysis capability from Day 1 applied to a cost question instead of a quality question.
- **Identify the costliest functions first** — a chatbot that calls a Use LLM-heavy Target Function on every single turn — the Fraud Detection Chatbot (Batch 1) calling FraudRiskAssessment on every claim it's asked about — is a natural first place to look for cost, before a function most of whose work is deterministic Object-category blocks.
- **Weigh cost against the judgment the Use LLM block actually buys you** — the fraud-judgment block inside FraudRiskAssessment exists because a deterministic threshold check alone (thresholdCrossed) cannot weigh fraudScore, claimSummary, and linkedInvestigations' findings together — the cost is the price of a genuine judgment call, not a block to eliminate reflexively. Cost management is about visibility and trade-off, not blanket reduction.

6. Worked Example: Where the Cost Actually Sits

Mapping CoverageRecommendationAssessment's (Batch 4) blocks to their cost profile:

- **Low-cost, deterministic blocks** — the Create object set and Filter blocks narrowing to thisRenewal, the Get object list element at index block extracting proposedPremium, renewalDate, and status, and the thresholdCrossed Create variable block checking proposedPremium against €5,000 — all run with no LLM call involved, at negligible marginal cost regardless of traffic volume.
- **High-cost, judgment blocks** — the Use LLM block weighing atRiskSignals, retentionRecommendation, and linkedRetentionOffer to produce recommendationTier is the one block in this function that consumes tokens on every call — and it is called once per policy assessed, so its cost scales directly with how often the Policy Recommendation Chatbot (Batch 4) is used.
- **The practical takeaway** — at renewal-season volume (Topic 4), the Use LLM block's cost is the one worth projecting and watching — the deterministic blocks around it are effectively free by comparison, and cost management effort should follow that asymmetry.

Platform Mechanic

Foundry Observability's confirmed per-resource metrics (success/failure counts, P95 execution duration, execution counts) already tell you how often a function runs. Layering token usage and per-call cost — surfaced by Foundry Observability for LLM calls and exported to your own dashboard — onto those same execution counts is how a cost-management discipline turns a usage number into a cost number.

Topic 3 — Release Strategy

7. Why a Changed Prompt Needs a Release Strategy

- **A new prompt version is a behavior change, same as a code deploy** — Day 1 covered recording and evaluating a new prompt version before it replaces the old one in production; release strategy is the next question — does the new version go to every user at once, or gradually?
- **Staged rollout is the same discipline ordinary application code already uses** — rolling out a new prompt version for the Compliance Chatbot (Batch 3) to a small slice of Batch 3's traffic before full rollout is a staged release — the same discipline a blue/green or canary deployment gives ordinary application code, applied to a prompt instead of a binary.
- **Passing Day 1's evals is necessary but not sufficient for a full rollout** — a new version of the Fraud Detection Chatbot's (Batch 1) system prompt that passes its Evaluation Suite's Test Cases has cleared the bar Day 1 described — release strategy adds a second bar: does it hold up against real traffic at small scale before it reaches everyone?

8. Elements of a Staged Release

- **A known-good version stays live while the new one is tested** — the prior, recorded version of RiskScoreRefresh's (Batch 2) judgment prompt — Day 1's prompt-versioning discipline — keeps serving most traffic while a revised version is evaluated, the same way a blue environment keeps serving while green is validated.
- **The new version is exposed to a deliberately small slice first** — routing a defined fraction of Underwriting Chatbot (Batch 2) conversations to a revised prompt version, rather than every conversation, limits the blast radius if the new version underperforms in ways the Evaluation Suite's Test Cases didn't anticipate.
- **A clear rollback path is defined before rollout starts, not improvised after** — if the EscalateForCoverageRecommendation handoff logic in a revised Customer Support Chatbot

(Batch 4) prompt starts firing incorrectly for real users, reverting to the previous recorded version — the same rollback Day 1's prompt versioning made possible — should be the predefined, immediate response.

9. Release Strategy in Practice: A Staged Rollout

Rolling out a revised task prompt for the Compliance Chatbot (Batch 3):

1. A revised system prompt is drafted to tighten when the chatbot escalates to `SubmitComplianceReview`

Following Day 1's versioning discipline, the change is recorded as a new, labeled version rather than overwritten silently over the prior one.

2. The new version is validated against `ComplianceReadinessCheck`'s Evaluation Suite before any user sees it

The same Test Cases used to validate the prior version, plus new ones covering the tightened escalation rule, run and their Metrics are compared against the prior version's.

3. The new version is released to a small slice of Batch 3's traffic first

A defined fraction of conversations route to the revised prompt while the rest continue on the known-good version — exactly the staged exposure a blue/green deployment gives ordinary code.

4. Observability and evals both confirm the new version before full rollout

Foundry Observability's confirmed metrics (failure rate, P95 execution duration) stay stable for the small slice, and a second evals run shows no regression — only then does the new version become the default for all of Batch 3.

Checkpoint — Confirm Before Continuing

Before moving on: for a chatbot you've built (Claims, Underwriting, Fraud Detection, Compliance, Risk Assessment, Customer Support, or Policy Recommendation), could you name who would see a revised prompt first if you released it in stages — and what signal would tell you it's safe to release to everyone?

Topic 4 — Capacity Planning

10. Why LLM-Powered Systems Need Capacity Planning

- **LLM-powered calls are slower and costlier per request than a classical lookup** — a surge in conversations with the Claims Chatbot (Batch 1) means a surge in calls to its `FraudRiskAssessment` tool's Use LLM block — Topic 2's cost discipline and this topic's load discipline are directly connected: more traffic means more cost and more load at the same time.
- **Foundry Observability's confirmed metrics are what you plan against** — P95 execution duration and execution counts for `FraudRiskAssessment`, tracked over the last 30 days, are the concrete baseline a capacity plan starts from — you cannot plan for a spike you have no baseline to compare it against.
- **Production teams commonly plan for known seasonal or event-driven surges** — this is a general, unnamed industry practice, not a Palantir-specific feature: production teams commonly model expected peak load ahead of a known surge window and validate that their systems hold up at that projected volume before the surge arrives, rather than discovering capacity limits during it.

11. Two Concrete Surge Scenarios From This Program

- **A claims surge after a catastrophic weather event (Batch 1)** — a regional storm or flood event would drive a sudden multiple of normal call volume into the Claims Chatbot and, downstream, into `FraudRiskAssessment`'s Use LLM block — exactly the kind of event-driven spike capacity

planning exists to anticipate, including whether the chatbot's response time (P95 execution duration) stays acceptable under that load.

- **A renewal-season surge (Batch 4)** — policy renewals cluster around predictable calendar windows — a surge in Customer Support Chatbot conversations and CoverageRecommendationAssessment calls during peak renewal season is foreseeable well in advance, unlike the Batch 1 weather scenario, which makes it a planning exercise rather than an incident response.
- **What both scenarios share** — in both cases, the deterministic blocks (thresholdCrossed, riskScoreStale-style checks) scale cheaply with volume, while the Use LLM judgment block is where load and cost both concentrate — the same asymmetry Topic 2 identified, now viewed through a capacity lens instead of a cost lens.

12. Building a Capacity Plan

- **Establish the current baseline first** — pull FraudRiskAssessment's (Batch 1) and the Claims Chatbot's execution counts and P95 execution duration from Foundry Observability's 30-day window as the starting point — you cannot size for 5x normal volume without knowing what normal volume and normal latency are today.
- **Project the expected surge multiple** — for Batch 4's renewal-season surge, estimate the expected multiple of normal renewal volume from known historical renewal-cycle patterns, then project what that multiple means for CoverageRecommendationAssessment call volume and the Policy Recommendation Chatbot's expected response time at that load.
- **Validate before the surge, not during it** — where possible, test the projected load against the actual function ahead of the surge window — a renewal-season or post-catastrophic-event capacity test run in advance is far cheaper than discovering a bottleneck live, when adjusters or policyholders are already waiting.

Topic 5 — AI Governance & Responsible AI

13. AI Governance Builds on Foundry Governance You Already Know

- **Roles — who can do what with an AI system** — the same Role-based access control from Week 1 that governs who can edit an Ontology object type governs who can edit a chatbot's system prompt, publish a new version of FraudRiskAssessment, or change an Action Type's confirmation setting — AI governance starts with the same question Week 1 asked about any Foundry asset: who has the Role to change this?
- **Markings — what sensitive data an AI system can touch** — Week 1's Markings (ABAC-style) already govern which users and services can see which data — applied to AI, a Marking on sensitive claimant or policyholder data determines whether a chatbot's Object Query tool, such as the Claims Chatbot's query access to Claim and Insurance Policy, can actually see Marked fields, not just whether a person could.
- **Security Policies — the AI-specific risk they need to cover** — a chatbot's Action tool is a write path into the Ontology, same as any Action a person submits by hand. The confirmation-required setting on EscalateForCoverageRecommendation (Batch 4) — a person must approve it before it writes — is a governance control a Security Policy should mandate for any AI-triggered write, not something to rely on the chatbot's builder having remembered to set.

14. An AI-Specific Risk: Unchecked Write Access

- **The risk is specific to AI, not generic to Actions** — a person submitting ApproveClaim (Batch 1) by hand reads the form before clicking submit. A chatbot proposing the same Action is acting on an LLM's judgment — the confirmation step is what keeps a person in that loop, and removing it

(or never checking whether it's still in place) hands an LLM's judgment a direct, unreviewed write path.

- **Governance has to check this deliberately, not assume it** — every governed write tool across this program's four batches — ApproveClaim, RequestRiskReview, SubmitComplianceReview, EscalateForCoverageRecommendation — was built with confirmation required. An AI governance review's job is to periodically confirm that setting is still in place, not to assume a setting configured months ago in Week 3 or Week 5 is still correct today.
- **Responsible AI means the chatbot's own instructions reinforce the same discipline** — the Compliance Chatbot's (Batch 3) system prompt instruction to never state a compliance read without having called ComplianceReadinessCheck first is a responsible-AI guardrail written into the prompt itself — governance at the platform level (Roles, Markings, confirmation) and governance at the prompt level work together, not as substitutes for each other.

15. Foundry Governance Primitives, Applied to AI

Primitive	Week 1 Purpose	AI-Specific Application
Roles	Who can edit an Ontology object type or dataset	Who can edit a chatbot's system prompt, publish a function, or change an Action's confirmation setting
Markings	Which users/services can see Marked sensitive data	Whether a chatbot's Object Query tool can surface Marked fields in its responses
Security Policies	Platform-wide rules governing access and change	Mandating confirmation-required on every AI-triggered Action, not leaving it to each builder's discretion
Data Lineage	Tracing a dataset's origin and transformations	Tracing which inputs fed a chatbot's recommendation (Topic 6)

Common Mistake

Giving a chatbot's Action tool unchecked write access because confirmation was set once and forgotten. The confirmation-required setting on EscalateForCoverageRecommendation (Batch 4) is a governance control, not a formality — treating it as checked-and-done rather than periodically reviewed is how an AI-specific write risk goes unnoticed.

Topic 6 — Audit & Compliance

16. Audit and Compliance Build on Foundry Primitives You Already Know

- **Data Lineage traces what fed into a decision** — the same Data Lineage from Week 1 that traces a dataset's origin and transformations can trace which Claim record, which linkedInvestigations results, and which ClaimsCopilot output fed into a specific FraudRiskAssessment (Batch 1) run — answering “what did this recommendation see?”, not just “what did it output?”.
- **Audit Trails record who did what, and when** — Week 1's Audit Trails, which already record who changed an Ontology object or published a function, extend naturally to recording when the Fraud Detection Chatbot proposed OpenInvestigation, when a person confirmed it, and when the Investigation object was actually created — a complete, timestamped record of an AI-initiated write.
- **Together, they answer the regulator's actual question** — “why did this chatbot take this action, on this record, at this time, and who allowed it?” is answered by combining Data Lineage (what fed the decision) with Audit Trails (who approved the resulting write) — neither alone tells the complete story.

17. Audit & Compliance in Practice: Tracing One Chatbot Action

Answering “why did the Compliance Chatbot (Batch 3) recommend ApproveReferral on this Fraud Case?”:

1. Start from the Audit Trail entry for the SubmitComplianceReview submission

It records who confirmed the Action, and when — the person-in-the-loop the confirmation-required setting (Topic 5) was there to guarantee.

2. Trace back through Data Lineage to the ComplianceReadinessCheck run that produced the recommendation

Identifying exactly which run — and therefore which prompt version, per Day 1's versioning discipline — generated the recommendedComplianceAction = ApproveReferral that the chatbot proposed.

3. Trace further back to the inputs that run actually used

The specific linkedEvidence and linkedReferralSource object sets, and the caseSummary and missingEvidence output from FraudTriageCopilot, that fed the Use LLM compliance-judgment block on that specific run.

4. The result is a complete, defensible answer

Not “the chatbot decided this,” but a traceable chain from the Fraud Case's actual evidence, through a specific function run, to a specific person's confirmation — exactly what an audit or regulatory inquiry needs, assembled from Foundry primitives already in place, not reconstructed from memory.

18. Tying Audit & Compliance Back to Responsible AI

- **Evaluation and observability (Day 1) establish that a function behaves correctly** — Data Lineage and Audit Trails (today) establish that you can prove, after the fact, exactly why it acted the way it did on one specific occasion — a different and equally necessary guarantee.
- **Drift monitoring, cost management, release strategy, and capacity planning keep the system healthy in operation** — governance and audit keep it accountable — together, all six of today's topics plus Day 1's four complete the production-readiness bar this week set out to establish.
- **This is the Week 6 deliverable's foundation** — the Production Readiness Package you build Wednesday — an evaluation suite, an observability dashboard, and a release plan — rests on exactly this combination: Day 1's evaluation and observability, and today's operational and governance disciplines, applied to one of your own functions or chatbots.

Takeaway

Audit and compliance are not a separate add-on at the end of a project — they depend on Data Lineage and Audit Trails being in place before a question is ever asked. The same discipline that makes governance real (Topic 5) is what makes an audit trail traceable (Topic 6).

19. Week 6 Deliverable Recap: The Production Readiness Package

- **An evaluation suite** — built on Day 1's AIP Evals concepts — Target Function, Test Cases, Evaluation Function, Metrics — re-run periodically per today's drift-monitoring discipline, not just once at launch.
- **An observability dashboard** — built on Day 1's Foundry Observability metrics, extended with today's cost-management view (token usage and cost per call) and a capacity-planning baseline (execution counts and P95 duration against a projected surge).
- **A release plan** — documenting today's staged-rollout approach for the next prompt or model change, the governance checks (Roles, Markings, confirmation settings) it must pass, and the Data Lineage and Audit Trail evidence it will leave behind.

20. Looking Ahead: Day 3 and the Rest of Week 6

- **Day 3 — Self-paced build** — you will build the evaluation suite, observability dashboard, and release plan described above for one of your program's existing functions or chatbots — today's six topics, applied hands-on, alongside Day 1's four.
- **Day 4 and 5 — Batch practicals and review** — working sessions and review of the Production Readiness Package each batch produces, closing out Week 6 and this program's production-readiness arc.

21. Common Pitfalls

- **Assuming a function that passed Day 1's evals last month is still performing the same way today** — a platform-level model update can shift a function's judgment with zero code or prompt change flagged — that is drift.
- **Treating every Use LLM block as costing the same as a deterministic block** — RiskScoreRefresh's (Batch 2) Use LLM risk-judgment block carries a real per-call token cost that its deterministic riskScoreStale calculation does not.
- **Rolling out a new prompt version to 100% of users at once** — replacing a chatbot's system prompt for every trainee overnight, with no staged rollout, means a regression is discovered by every user at once instead of by a small slice first.
- **Treating capacity planning as a one-time sizing exercise** — a renewal-season surge is a predictable seasonal pattern, not a one-off — capacity planning that isn't revisited before each known surge window is already stale by the time it's needed.
- **Giving a chatbot's Action tool unchecked write access because confirmation was set once and forgotten** — the confirmation-required setting on a governed Action is a governance control, not a formality.
- **Assuming audit and compliance can be reconstructed after the fact from memory** — tracing why a chatbot called a governed Action on a specific record depends on Data Lineage and Audit Trails being in place before the question is asked, not after a regulator asks it.

22. Glossary

Term	Meaning
Drift	A shift in an LLM-powered function's output quality or behavior with no visible code or prompt change — detected by re-running AIP Evals and watching Foundry Observability trends over time
Cost Management	The LLMOps discipline of tracking token usage and per-call cost, concentrated in Use LLM blocks rather than deterministic blocks, and establishing a dashboard to watch it as traffic scales
Release Strategy	A staged rollout of a new prompt or model version to a small slice of traffic before full release, mirroring a blue/green or canary deployment for ordinary code
Capacity Planning	Establishing a current baseline from Foundry Observability metrics and projecting it forward against an expected traffic or load surge, validated before the surge arrives
AI Governance	Applying Foundry's confirmed governance primitives (Roles, Markings, Security Policies) to AI-specific risks, such as a chatbot's unchecked Action write access
Responsible AI	Guardrails written directly into a chatbot's own system prompt (such as never acting without first calling its reasoning tool) that reinforce platform-level governance
Data Lineage	Tracing what inputs — records, linked object sets, upstream function outputs — fed a specific function run or chatbot recommendation
Audit Trail	A timestamped record of who did what and when, including who confirmed an AI-proposed Action and when the resulting write occurred
Production Readiness Package	The Week 6 deliverable: an evaluation suite, an observability dashboard, and a release plan, built Day 3 on Day 1 and Day 2's combined foundations

Reflection Questions

- Pick one function or chatbot you've built in this program. What would a drift-monitoring re-run schedule for its Evaluation Suite realistically look like, and what Foundry Observability trend would you watch alongside it?
- For the same function, which of its blocks are deterministic and which involve a Use LLM call? Where would cost concentrate if traffic to it tripled?
- If you were releasing a revised prompt for one of your batch's chatbots, who would see it first, and what would tell you it's safe to release to everyone else?
- Think of a traffic surge your batch's scenario would realistically face (a catastrophic event for Claims, a renewal season for Policy Servicing, or an equivalent for Underwriting or Fraud/SIU). What would you need to know about today's baseline to plan for it?
- For a governed Action you've built (ApproveClaim, RequestRiskReview, SubmitComplianceReview, or EscalateForCoverageRecommendation), who currently has the Role to change its confirmation setting — and when did you last check?

Further Practice

Continue into Day 3, the self-paced build, where you will assemble your own Production Readiness Package: an evaluation suite, an observability dashboard, and a release plan for one of your program's existing functions or chatbots, drawing on both Day 1's and today's foundations.